Universität Augsburg
Mathematisch-Naturwissenschaftlich-Technische Fakultät

Applications of static and measurement based code analysis

- From computational physics to embedded systems programming -

Jonas Kell
University of Augsburg
Chair for theoretical Physics III

28th of January 2026

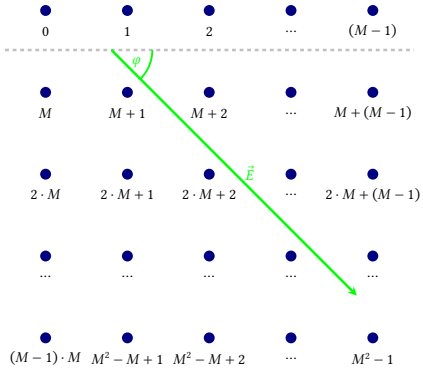
What to expect from this talk

- Jonas Kell
  - University of Augsburg, Bavaria Germany
- Bachelor of Physics
- Bachelor of Computer Science
- Master of Physics
  - Minor in Computer Science
  - Focus on theoretical, computational many-body Physics

What does a theoretical Physicist do?

Introduction: Work of a computational Physicist

- Condensed matter Physics
  - Particles at discrete positions on a lattice
  - Interaction governed by the *Hamiltonian*
  - Particle-type has influence on the behavior



What does a theoretical Physicist do?

Introduction: Work of a computational Physicist

$$\left[\hat{h}_l, \hat{h}_m\right] = 0$$
$$\left[\hat{h}_l^\dagger, \hat{h}_m^\dagger\right] = 0$$
$$\left[\hat{h}_l, \hat{h}_m^\dagger\right] = \left(1 - 2 \cdot \hat{h}_m^\dagger \hat{h}_m\right) \cdot \delta_{l,m}$$

$$\left(\hat{h}_l^\dagger \hat{h}_l\right)^n = \hat{h}_l^\dagger \hat{h}_l \qquad \forall n \in \mathbb{N}$$

$$\hat{h}_m^{\dagger \mathrm{I}}(t) = e^{i \cdot \varepsilon_m \cdot t} \left(1 + \left(e^{i \cdot U \cdot t} - 1\right) \hat{d}_m^{\dagger \mathrm{S}} \hat{d}_m^{\mathrm{S}}\right) \hat{h}_m^{\dagger \mathrm{S}}$$
$$\hat{h}_m^{\mathrm{I}}(t) = e^{-i \cdot \varepsilon_m \cdot t} \left(1 + \left(e^{-i \cdot U \cdot t} - 1\right) \hat{d}_m^{\dagger \mathrm{S}} \hat{d}_m^{\mathrm{S}}\right) \hat{h}_m^{\mathrm{S}}$$
$$\hat{d}_m^{\dagger \mathrm{I}}(t) = e^{i \cdot \varepsilon_m \cdot t} \left(1 + \left(e^{i \cdot U \cdot t} - 1\right) \hat{h}_m^{\dagger \mathrm{S}} \hat{h}_m^{\mathrm{S}}\right) \hat{d}_m^{\dagger \mathrm{S}}$$
$$\hat{d}_m^{\mathrm{I}}(t) = e^{-i \cdot \varepsilon_m \cdot t} \left(1 + \left(e^{-i \cdot U \cdot t} - 1\right) \hat{h}_m^{\dagger \mathrm{S}} \hat{h}_m^{\mathrm{S}}\right) \hat{d}_m^{\mathrm{S}}$$

What does a theoretical Physicist do?

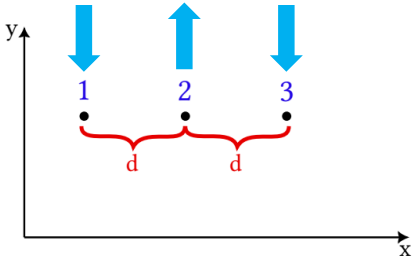
Introduction: Work of a computational Physicist

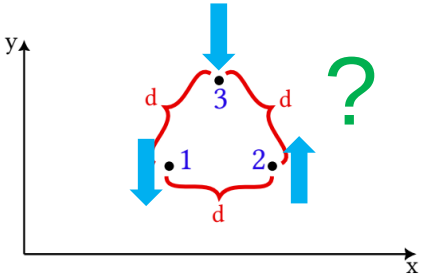
$$
\begin{aligned}
\hat{V}^{\mathrm{I}}(t) &= \left\{\hat{V}^{\mathrm{S}}\right\}^{\mathrm{I}}(t) = -J \cdot \sum_{[l,m]} \left\{ \left(\hat{h}_l^{\dagger \mathrm{S}} \hat{h}_m^{\mathrm{S}} + \hat{d}_l^{\dagger \mathrm{S}} \hat{d}_m^{\mathrm{S}} \right) \right\}^{\mathrm{I}}(t) \\
&= -J \cdot \sum_{[l,m]} \left(\hat{h}_l^{\dagger \mathrm{I}}(t) \hat{h}_m^{\mathrm{I}}(t) + \hat{d}_l^{\dagger \mathrm{I}}(t) \hat{d}_m^{\mathrm{I}}(t) \right) \\
&= -J \cdot \sum_{[l,m]} \left[\Lambda_{\mathrm{A}}(l,m,t) \cdot \hat{\mathrm{F}}_{\mathrm{A}}(l,m) + \Lambda_{\mathrm{B}}(l,m,t) \cdot \hat{\mathrm{F}}_{\mathrm{B}}(l,m) + \Lambda_{\mathrm{C}}(l,m,t) \cdot \hat{\mathrm{F}}_{\mathrm{C}}(l,m) \right]
\end{aligned}
$$

$$
\begin{aligned}
\Lambda_{\mathrm{A}}(l,m,t) &= e^{i \cdot (\varepsilon_l - \varepsilon_m) \cdot t} & \hat{\mathrm{F}}_{\mathrm{A}}(l,m) &= \sum_{\sigma \in \{\uparrow,\downarrow\}} \hat{h}_{l,\sigma}^{\dagger \mathrm{S}} \hat{h}_{m,\sigma}^{\mathrm{S}} \left(1 + 2 \cdot \hat{n}_{l,\bar{\sigma}}^{\mathrm{S}} \hat{n}_{m,\bar{\sigma}}^{\mathrm{S}} - \hat{n}_{l,\bar{\sigma}}^{\mathrm{S}} - \hat{n}_{m,\bar{\sigma}}^{\mathrm{S}} \right) \\
\Lambda_{\mathrm{B}}(l,m,t) &= e^{i \cdot (\varepsilon_l - \varepsilon_m + U) \cdot t} & \hat{\mathrm{F}}_{\mathrm{B}}(l,m) &= \sum_{\sigma \in \{\uparrow,\downarrow\}} \hat{h}_{l,\sigma}^{\dagger \mathrm{S}} \hat{h}_{m,\sigma}^{\mathrm{S}} \left(\hat{n}_{l,\bar{\sigma}}^{\mathrm{S}} - \hat{n}_{l,\bar{\sigma}}^{\mathrm{S}} \hat{n}_{m,\bar{\sigma}}^{\mathrm{S}} \right) \\
\Lambda_{\mathrm{C}}(l,m,t) &= e^{i \cdot (\varepsilon_l - \varepsilon_m - U) \cdot t} & \hat{\mathrm{F}}_{\mathrm{C}}(l,m) &= \sum_{\sigma \in \{\uparrow,\downarrow\}} \hat{h}_{l,\sigma}^{\dagger \mathrm{S}} \hat{h}_{m,\sigma}^{\mathrm{S}} \left(\hat{n}_{m,\bar{\sigma}}^{\mathrm{S}} - \hat{n}_{m,\bar{\sigma}}^{\mathrm{S}} \hat{n}_{l,\bar{\sigma}}^{\mathrm{S}} \right)
\end{aligned}
$$

Where Computer Science comes into Play

Introduction: Work of a computational Physicist

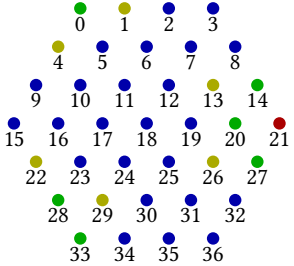




Where Computer Science comes into Play

Introduction: Work of a computational Physicist

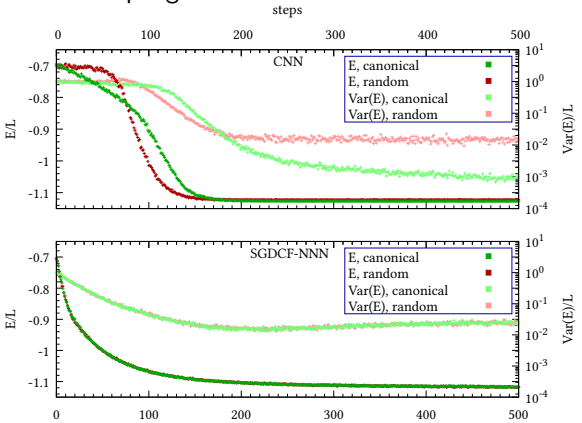
$$
\begin{array}{c}
 \\
\langle\uparrow,\uparrow,\uparrow| \\
\langle\uparrow,\uparrow,\downarrow| \\
\langle\uparrow,\downarrow,\uparrow| \\
\langle\uparrow,\downarrow,\downarrow| \\
\langle\downarrow,\uparrow,\uparrow| \\
\langle\downarrow,\uparrow,\downarrow| \\
\langle\downarrow,\downarrow,\uparrow| \\
\langle\downarrow,\downarrow,\downarrow|
\end{array}
\begin{array}{c}
\begin{array}{cccccccc}
|\uparrow,\uparrow,\uparrow\rangle & |\uparrow,\uparrow,\downarrow\rangle & |\uparrow,\downarrow,\uparrow\rangle & |\uparrow,\downarrow,\downarrow\rangle & |\downarrow,\uparrow,\uparrow\rangle & |\downarrow,\uparrow,\downarrow\rangle & |\downarrow,\downarrow,\uparrow\rangle & |\downarrow,\downarrow,\downarrow\rangle
\end{array} \\
\begin{pmatrix}
3\cdot J & h & h & 0 & h & 0 & 0 & 0 \\
h & -1\cdot J & 0 & h & 0 & h & 0 & 0 \\
h & 0 & -1\cdot J & h & 0 & 0 & h & 0 \\
0 & h & h & -1\cdot J & 0 & 0 & 0 & h \\
h & 0 & 0 & 0 & -1\cdot J & h & h & 0 \\
0 & h & 0 & 0 & h & -1\cdot J & 0 & h \\
0 & 0 & h & 0 & h & 0 & -1\cdot J & h \\
0 & 0 & 0 & h & 0 & h & h & 3\cdot J
\end{pmatrix}
\end{array}
$$



Modern Methods instead of brute-force Computation

Machine-Learning for Physics

- Utilization of the *universal approximator* property of neural networks
  - "Training" on some states to obtain approximate solutions for all
  - Monte-Carlo sampling



Type Annotations and strongly typed Systems

Machine-Learning for Physics

```python
class Metaformer(nn.Module):
    """Metaformer architecture, based on the experiments conducted for graph-image-processing

    Initialization arguments:
        * ``lattice_parameters``: Info on the shape of the input used for patch-embedding

    """

    lattice_parameters: LatticeParameters
    embed_dim: int = 60  # ! must be divisible by num_heads
    embed_mode: Literal[
        "duplicate_spread", "duplicate_nn", "duplicate_nnn"
    ] = "duplicate_spread"
    depth: int = 12
    mlp_ratio: float = 4.0
    norm_layer: Callable = nn.LayerNorm
    act_layer: Callable = nn.gelu
    # token mixing
    token_mixer: Literal[
        "attention",
        "pooling",
        "convolution",
    ] = "attention"
    mixing_symmetry: Literal["arbitrary", "symm_nn", "symm_nnn"] = "arbitrary"
    # attention specific arguments
    num_heads: int = 6
    qkv_bias: bool = True
    ansatz: Literal[
        "single-real", "single-complex", "split-complex", "two-real"
    ] = "single-real"
    head: Literal["act-fun", "cnn"] = "act-fun"
```

Type Annotations and strongly typed Systems

Machine-Learning for Physics

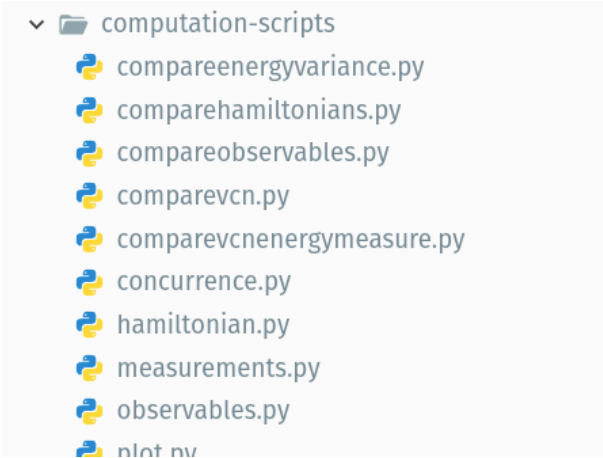
- Type annotations with the `typing` package
- Python linting settings
- (Doc) comments

```python
# get model + Variational wave function
seed = 1234
if ansatz in ["single-real", "single-complex", "split-complex"]:
    model = model_fn(
        lattice_parameters, depth, embed_dim, num_heads, mlp_ratio, ansatz
    )
    psi = jVMC.vqs.NQS(model, seed=seed, batchSize=nqs_batch_size)
elif ansatz == "two-real":
    model1 = model_fn(
        lattice_parameters, depth, embed_dim, num_heads, mlp_ratio, ansatz
    )
    model2 = model_fn(
        lattice_parameters, depth, embed_dim, num_heads, mlp_ratio, ansatz
    )
    psi = jVMC.vqs.NQS((model1, model2), seed=seed, batchSize=nqs_batch_size)
else:
    raise RuntimeError(f"Ansatz '{ansatz}' ist not supported")
```

```
(parameter) model_fn: (LatticeParameters, int, int, int, int, Literal['single-real', 'single-complex', 'split-complex', 'two-real']) ->
    Any
```

Test driven Development

Machine-Learning for Physics



Analytical Calculations with Math-Manipulator

A personalized computer algebra system

- Built from the need to re-order non-commutative operator strings
- Open Source and web based
- Possibility to practice static code analysis and compiler components

Available here

https://jonas-kell.github.io/math-manipulator/

Analytical Calculations with Math-Manipulator

A personalized computer algebra system

Distribute and cancel from sums

The handy "Distribute" function can be used to expand products of sums.
After the terms have been fully expanded, the "Eliminate Canceling Terms" function of sums is kicked into action.
In this case "Order Operator Strings" needed to be executed twice manually (the author of the tool mainly uses it for non-commutative operators, so ordering per default is not such a clean idea).

```
(x+y)(3+z)(x-y)
```

$$((x+y)\cdot(3+z)\cdot(x-y))$$

Sel. Parent | Sel. First Child | Sel. Previous Sibling | Sel. Next Sibling

Replace | Replace All Equal | Replace (With Export) | Define Variable | Order Operator Strings | Fold Numbers | Cleanup Terms | Distribute | Multiply Complex Numbers | Pull Out Minus
Merge And Evaluate Bra Ket | Rename With Swap | Pack Same Variables | Replace All With | Elements Cancel | Adjacent Elements Cancel

Selected Export to Clipboard | Show Export Structure (UUIDs) | Show Export Structure | Show Latex

$$((x\cdot 3\cdot x)+(x\cdot 3\cdot -y)+(x\cdot z\cdot x)+(x\cdot z\cdot -y)+(y\cdot 3\cdot x)+(y\cdot 3\cdot -y)+(y\cdot z\cdot x)+(y\cdot z\cdot -y))$$

Sel. Parent | Sel. First Child | Sel. Previous Sibling | Sel. Next Sibling

Replace | Replace All Equal | Replace (With Export) | Define Variable | Commute with subsequent | Order Operator Strings | Fold Numbers | Cleanup Terms | Distribute | Multiply Complex Numbers
Pull Out Minus | Merge And Evaluate Bra Ket | Rename With Swap | Pack Same Variables | Replace All With | Elements Cancel | Adjacent Elements Cancel

Selected Export to Clipboard | Show Export Structure (UUIDs) | Show Export Structure | Show Latex

$$((3\cdot x\cdot x)-(3\cdot x\cdot y)+(x\cdot z\cdot x)-(x\cdot z\cdot y)+(3\cdot y\cdot x)-(3\cdot y\cdot y)+(y\cdot z\cdot x)-(y\cdot z\cdot y))$$

Sel. Parent | Sel. First Child | Sel. Previous Sibling | Sel. Next Sibling

Replace | Replace All Equal | Replace (With Export) | Define Variable | Commute with subsequent | Order Operator Strings | Fold Numbers | Cleanup Terms | Distribute | Multiply Complex Numbers
Pull Out Minus | Merge And Evaluate Bra Ket | Rename With Swap | Pack Same Variables | Replace All With | Elements Cancel | Adjacent Elements Cancel

Selected Export to Clipboard | Show Export Structure (UUIDs) | Show Export Structure | Show Latex

$$((3\cdot x\cdot x)+(x\cdot z\cdot x)-(x\cdot z\cdot y)-(3\cdot y\cdot y)+(y\cdot z\cdot x)-(y\cdot z\cdot y))$$

Analytical Calculations with Math-Manipulator

A personalized computer algebra system

Sort the summands in a sum

Not only is it possible to order products, but also sums with the "Order Summands" action.
The program tries to order helpfully, but this is not always easy.
For better visibility e.g. negations of operators will be sorted always right to the right of the operator.

```
(x*y)+(x*y*z)+(y*y)-(x*y)+z+x+(z*z*z)
```

$$((x \cdot y) + (x \cdot y \cdot z) + (y \cdot y) - (x \cdot y) + z + x + (z \cdot z \cdot z))$$

Sel. Parent | Sel. First Child | Sel. Previous Sibling | Sel. Next Sibling

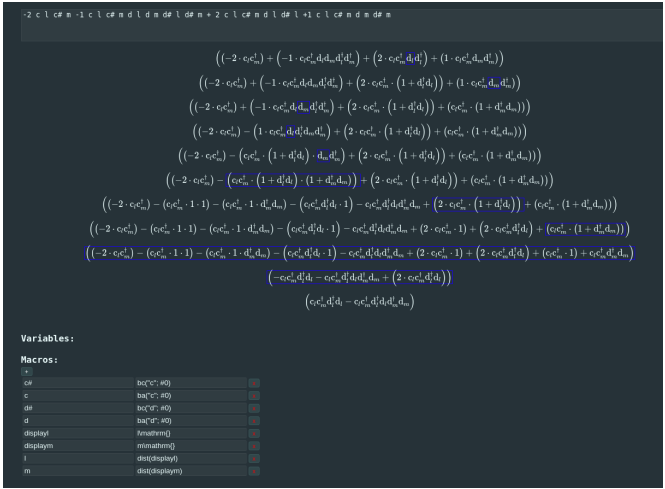
Replace | Replace All Equal | Replace (With Export) | Define Variable | Fold Numbers | Cleanup Terms | Pull Out Minus | Eliminate Canceling Terms | Combine Complex Numbers
Order Summands | Factor Out Left | Factor Out Right | Group Equal Elements | All Summands Order Operator String | Rename With Swap | Pack Same Variables
Replace All With | Elements Cancel | Adjacent Elements Cancel

Selected Export to Clipboard | Show Export Structure (UUIDs) | Show Export Structure | Show Latex

$$((x \cdot y \cdot z) + (y \cdot y) + z + x + (z \cdot z \cdot z))$$

Analytical Calculations with Math-Manipulator

A personalized computer algebra system



What is “optimal”? - Quiz of Intuition

Code optimization through measurements

```python
def part_A_flipping(
    Lu: int, Ld: int, Mu: int, Md: int, flip_up: bool, flip_l: bool
) -> int:

    if flip_up:
        if flip_l:
            Lup = not Lu
            Ldp = Ld
            Mup = Mu
            Mdp = Md
        else:
            Lup = Lu
            Ldp = Ld
            Mup = not Mu
            Mdp = Md
    else:
        if flip_l:
            Lup = Lu
            Ldp = not Ld
            Mup = Mu
            Mdp = Md
        else:
            Lup = Lu
            Ldp = Ld
            Mup = Mu
            Mdp = not Md

    res = 0
    res += Lu and not Mu and Ld == Md
    res -= Lup and not Mup and Ldp == Mdp
    res += Ld and not Md and Lu == Mu
    res -= Ldp and not Mdp and Lup == Mup
```

What is “optimal”? - Quiz of Intuition

Code optimization through measurements

```python
def part_A_flipping_if_free(
    Lu: int, Ld: int, Mu: int, Md: int, flip_up: bool, flip_l: bool
) -> int:

    Lup = Lu == (not (flip_up and flip_l))
    Mup = Mu == (not flip_up or flip_l)  # (not (flip_up and not flip_l))
    Ldp = Ld == (flip_up or not flip_l)  # (not (not flip_up and flip_l))
    Mdp = Md == (flip_up or flip_l)  # (not (not flip_up and not flip_l))

    res = 0
    res += Lu and not Mu and Ld == Md
    res -= Lup and not Mup and Ldp == Mdp
    res += Ld and not Md and Lu == Mu
    res -= Ldp and not Mdp and Lup == Mup

    return res
```

What is “optimal”? - Quiz of Intuition

Code optimization through measurements

```python
def part_A_flipping(
    Lu: int, Ld: int, Mu: int, Md: int, flip_up: bool, flip_l: bool
) -> int:
    res = 0
    res += (
        (Lu and not Ld and not Mu and not Md)
        or (not Lu and Ld and not Mu and not Md)
        or (Lu and Ld and Mu and not Md)
        or (Lu and Ld and not Mu and Md)
    )
    res -= (
        (Lu and not Ld and not Mu and Md and not flip_up)
        or (not Lu and Ld and not Mu and Md and not flip_up and not flip_l)
        or (Lu and Ld and Mu and Md and not flip_l)
        or (Lu and not Ld and Mu and not Md and flip_up and not flip_l)
        or (not Lu and Ld and Mu and not Md and flip_up)
        or (not Lu and not Ld and not Mu and not Md and flip_l)
        or (Lu and not Ld and Mu and not Md and not flip_up and flip_l)
       or (not Lu and Ld and not Mu and Md and flip_up and flip_l)
        or (Lu and Ld and not Mu and not Md)
    )
    return res
```

What is “optimal”? - Answer

Code optimization through measurements

Boolean Logic optimized	23.12 s
Branch free optimized	22.02 s
Default implementation	18.8 s

- Nested If-s are indeed the fastest...
  - Might seem un-intuitive. Intuition may be wrong!
  - But probably not for every language / workload
  - Measurements required per use-case!

Why pivot to Computer Science?

Lookout: Timing Analyzable Rust Compilation

- Basic lessons for a Physicist from a Computer Scientist
  - Prefer strongly typed systems
  - Perform system-specific measurements
  - Test early in and as part of the development cycle
  - Avoid boilerplate, use automatic code generation
  - Leverage static code analysis wherever possible
- I prefer developing the tools over using them
  - Start of a research proposition (initially in Augsburg)

The Problem: WCET

Lookout: Timing Analyzable Rust Compilation

- WCET: *Worst Case Execution Time*
- High-volume computation mainly requires low average runtimes
  - Rare longer-running processes are not that bad
- Not so in embedded computing!
- Example: Airbag deployment system
  - Not relevant, whether 1ns or 5ns
  - BUT, we must guarantee execution faster than a certain upper bound

Advantages of Rust

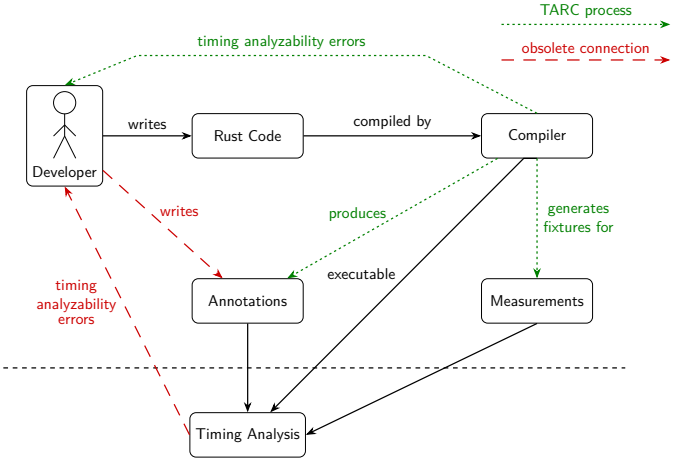
Lookout: Timing Analyzable Rust Compilation

```c
typedef struct
{
    int x;
} Small;

int main()
{
    Small arr[3] = {{1}, {2}, {3}};

    Small *ptr = (Small *)((char *)arr + sizeof(Small));

    printf("Second element: %d\n", ptr->x); // prints 2

    return 0;
}
```

```rust
#[derive(Debug)]
struct Small {
    x: i32,
}

fn main() {
    let arr = [Small { x: 1 }, Small { x: 2 }, Small { x: 3 }];

    let second = &arr[1];

    println!("Second element: {}", second.x); // prints 2
}
```

TARC: Infrastructure

Lookout: Timing Analyzable Rust Compilation

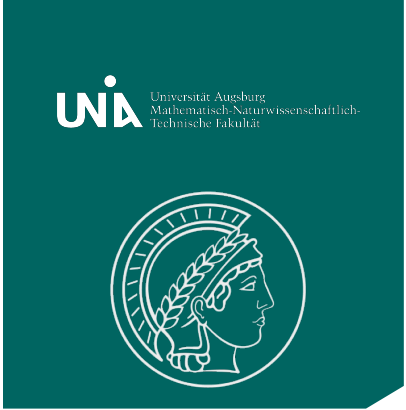


TARC: Minimal Example

Lookout: Timing Analyzable Rust Compilation

```rust
#[ta_analyzable()]
fn check_airbag_trigger_threshold() -> usize {
    let mut sensor_values = vec![0usize; NUM_SENSORS];

    for i in 0..NUM_SENSORS {
        // first argument in ta_bounded is the code to insert,
        // second one is a maximum runtime from an external source
        // (measurement, analysis, calculation etc.)
        sensor_values[i] = ta_bounded!({ read_sensor_value(i) }, 45);
    }

    let mut count = 0usize;
    for &value in &sensor_values {
        if value > THRESHOLD {
            count += 1;
        }
    }

    count
}
```

Thank you for your kind attention



Jonas Kell
jonas-kell@web.de
www.github.com/jonas-kell

References I

Summary & Conclusion

[1] J. Kell, *Classical networks for the hubbard model with a tilted potential*, (Feb. 13, 2025) https://github.com/jonas-kell/master-thesis-documents.

[2] J. Kell, *Investigation of transformer architectures for geometrical graph structures and their application to two-dimensional spin systems*, (Oct. 7, 2022) https://github.com/jonas-kell/bachelor-thesis-documents.

[3] J. Kell, *Math-manipulator tool - info and repository*, (2024) https://github.com/jonas-kell/math-manipulator.

Extra Slides: Overview

Extra slides

- Rust Macro Test Fixtures
- Rust Macro Compile Time Errors
- Results Master Thesis
- More Math-Manipulator Features
- TARC: Timeline
- Abstract

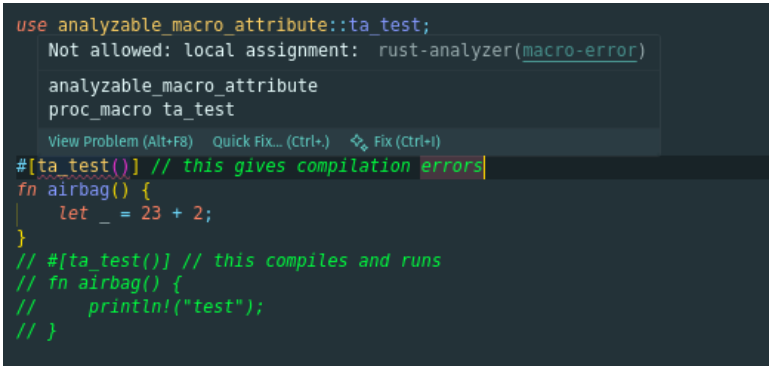
Extra Slide: Rust Macro Test Fixtures

Extra slides

```
$ cargo run
Compiling rust-test v0.1.0 (/home/jonas/github/rust-test)
    Finished ‘dev‘ profile [unoptimized + debuginfo] target(s) in 0.17s
    Running ‘/home/jonas/github/rust-test/target/debug/rust-test‘
Entering function: check_airbag_trigger_threshold
Entering Bounded Block
Bounded Block produced Result: 1000
Exiting Bounded Block after max runtime: 45
Entering Bounded Block
Bounded Block produced Result: 1500
Exiting Bounded Block after max runtime: 45
Entering Bounded Block
....
Bounded Block produced Result: 3500
Exiting Bounded Block after max runtime: 45
Exiting function: check_airbag_trigger_threshold
```

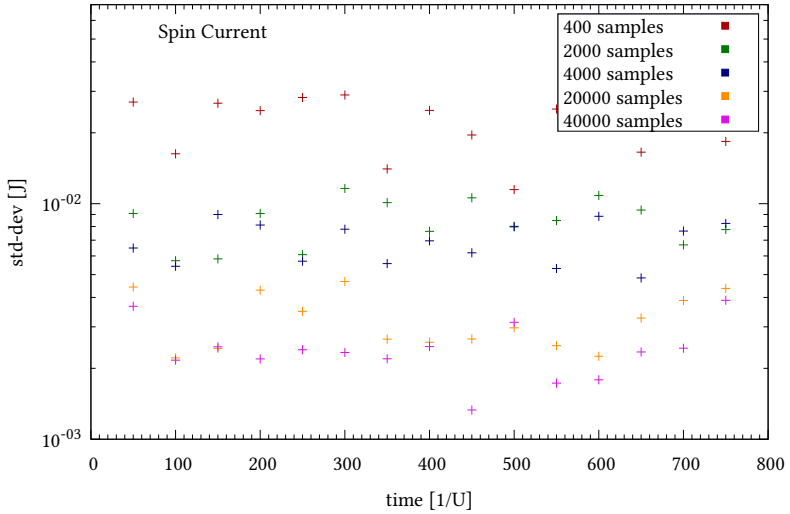
Extra Slide: Rust Macro Compile Time Errors

Extra slides



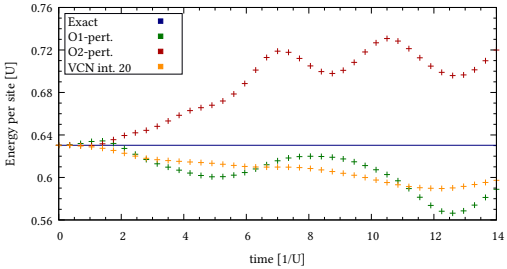
Extra Slide: Results Master Thesis

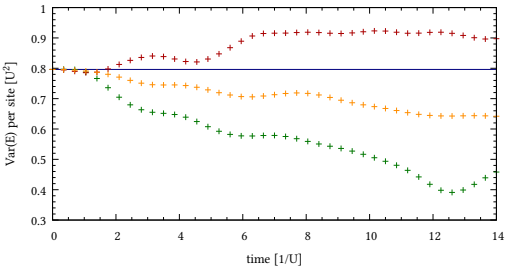
Extra slides



Extra Slide: Results Master Thesis

Extra slides





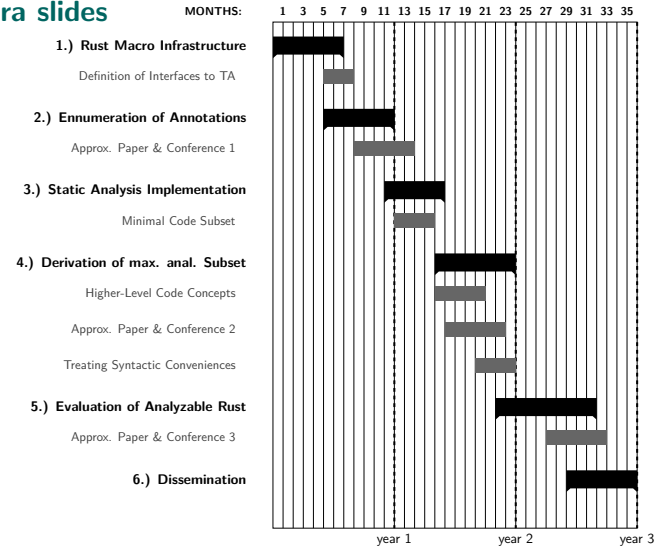
Extra Slide: More Math-Manipulator Features

Extra slides

- Intelligent text parsing
- Pice-wise hierarchical selection from GUI
- Function detection and custom definitions
- Constant detection
- Numerical values and variables
- Numerical constant folding
- User-definable macros
- Live variable declaration
- Bra-Ket evaluation (Physics)
- Creation- and Annihilation Operator support (Physics)
- Ordering, grouping, simplification
- Factoring and distributing
- Complex Numbers
- Rename or swap names
- Delta function evaluation
- Full LaTeX support...

Extra Slide: TARC: Timeline

Extra slides



Extra Slide: Abstract

Extra slides

When learning to write code, either overall or in a previously unknown programming language, many developers might get tripped up by the colors of text and instructions in their IDE. Fresh programmers might express typical questions like "Why are the words colored all of a sudden" or "If the compiler knows that there is a semicolon missing in line 12, why do I have to place it?". But even coding veterans might stumble upon the occasional odd "Why would this not work?" and get saved from hours of head-wrenching debugging, by the fact that the one variable is colored differently than the rest. A static code analysis tool just alerted them of the fact, that this identifier seems to be a reserved keyword in the language they were just tasked to work in.

Extra Slide: Abstract

Extra slides

With a brief introduction to the programming work a computational physicist might get confronted with, we want to explore possible uses for code analysis tooling and methods to improve the development of machine learning architectures or speed up the evaluation of expensive computational tasks.
We also want to pose the question: "If there is syntax checking and automatic refactoring for code, why not for doing your algebraic derivations?". That is what a computer algebra system is for! And it basically does the same: code analysis, just on your mathematical formulas.
This train of thought motivates our research proposal called *Timing Analyzable Rust Compilation*, that suggests a toolchain targeted at improving the timing analyzability of code for embedded systems. By providing feedback about timing analyzability already during the compilation step, we plan to streamline the code for downstream tools while it is still in the high-level language. All while generating annotation symbols and meta information for external calculations of the *Worst Case Execution Time* (WCET) or ensuring your functions are secure against timing-based side-channel attacks.